

RESEARCH ARTICLE

Android Malware Detection Method Combining Multi-Frequency Features and Convolutional Neural Networks

JIAN WANG^{1,2}, XUE HUA LIU³, MENG YUAN HUANG¹, PEI YI ZHOU¹, AND YUAN XU^{1,4}¹Faculty of Cyberspace Security, Software Engineering Institute of Guangzhou, Guangzhou 510900, China²Faculty of Applied Sciences, Macao Polytechnic University, Macao SAR, CO 999078, China³Faculty of Software and Artificial Intelligence, Software Engineering Institute of Guangzhou, Guangzhou 510900, China⁴College of Artificial Intelligence, Dongguan City University, Dongguan 523419, China

Corresponding author: Jian Wang (jenseWang@outlook.com)

This work was supported in part by the Projects of Universities in Guangdong Province, in 2024, under Grant 2024KTSCX133; and in part by the 2024 Guangdong Province Science and Technology Innovation Strategy Special Fund under Grant pdjh2024a493.

ABSTRACT With the rapid evolution of Android malware variants, traditional detection methods exhibit significant limitations in addressing complex patterns and adversarial attacks. This paper proposes a dual-modal feature extraction and fusion approach that integrates Convolutional Neural Networks (CNN) with Fourier frequency domain analysis for Android malware detection. Firstly, a novel feature extraction framework based on frequency domain analysis is constructed, which dynamically adjusts the boundary between high-frequency and low-frequency components through adaptive frequency selection. This framework effectively captures both the global patterns and local details of malware, significantly enhancing the robustness of frequency domain features. Secondly, a bidirectional recursive optimization-based feature fusion mechanism is designed, enabling deep integration of high-frequency, low-frequency, and spatial domain features through multiple rounds of iterative interaction. Experimental results demonstrate that the proposed method outperforms existing approaches in terms of accuracy, recall, and adversarial sample detection capability. Notably, it exhibits outstanding stability and performance in handling malware variants, offering an efficient and reliable technical framework for Android malware detection.

INDEX TERMS Adversarial android malware detection, convolutional neural networks (CNN), cyber security, dual-modal features, feature fusion, frequency domain analysis.

I. INTRODUCTION

With the widespread adoption of smartphones, the Android system has become the most widely used mobile operating system globally [1]. However, the openness and extensive application of the Android system has also made it a primary target for malware attacks. Recent reports indicate that over 3.5 million new Android malware variants were detected in 2023 alone [2]. The number of Android malware instances has been rapidly increasing annually, leading to security issues such as privacy breaches, data corruption, and system performance degradation [3], [4]. The continuous evolution of malware types and attack techniques has further increased the difficulty of malware detection. SH hammed et al. showed

in the literature the method of tapping potential in the field of autism detection by integrating traditional feature selection and machine learning technology [5]. Similarly, in the field of malware detection, with the development of deep learning, we are also exploring the combination of a variety of technologies to improve the detection ability.

Consequently, developing robust and highly accurate malware detection methods has become a research hotspot [6], [7]. Currently, Android malware detection primarily relies on signature-based methods, static analysis, and dynamic analysis. However, these traditional methods have shown limitations in the face of rapidly evolving malware variants and adversarial attacks [8].

Signature-based detection involves matching samples to known malware signature characteristics, such as hash values and characteristic instruction sequences [9]. While signature

The associate editor coordinating the review of this manuscript and approving it for publication was Vicente Alarcon-Aquino^{1b}.

detection achieves high accuracy, its limitation lies in its inability to identify new or slightly modified malware variants [10], [11]. Static analysis identifies malicious behaviors by parsing the APK files' code structure, resource files, and permission configurations. Common approaches include data flow analysis, control flow analysis, and permission combination analysis [12]. Although static analysis is fast, it can be easily bypassed using techniques such as obfuscation, encryption, or data padding, making it ineffective against complex malware samples. Dynamic analysis, on the other hand, executes applications in a sandbox or virtual environment to monitor behaviors such as system calls and network activities for malicious indicators [13], [14]. Despite its ability to capture runtime behaviors of malware, dynamic analysis incurs high computational costs and is susceptible to evasion techniques, such as delayed execution of malicious code or detection of simulation environments [15].

With the breakthroughs of deep learning in fields such as image processing and natural language processing, researchers have started integrating deep learning into malware detection, particularly by leveraging Convolutional Neural Networks (CNN) to process malware data in image form. This method transforms malware binary data into grayscale images, enabling CNNs to automatically extract features and classify samples. For instance, Kolosnjaji et al. proposed a method that converts malware bytecode into grayscale images and applies CNNs for detection, demonstrating the ability of CNNs to identify spatial domain features of malware [16]. However, CNN-based methods primarily focus on spatial domain features and struggle to recognize frequency domain features, such as padded data and periodic patterns [17].

Based on observations of Android malware structures and feature patterns, this paper proposes a dual-modal malware detection method that combines Fourier analysis with CNNs. The main contributions of this paper can be summarized as follows:

- 1) Introducing frequency domain features for malware detection: A novel feature extraction framework based on frequency domain analysis is proposed.
- 2) Designing an adaptive frequency selection method: By dynamically adjusting the boundary between high-frequency and low-frequency components, this method more accurately captures the global patterns and local details of malware, enhancing the robustness of feature extraction.
- 3) Proposing a bidirectional recursive optimization-based feature fusion approach: Through iterative interaction, this method deeply integrates high-frequency, low-frequency, and spatial domain features, significantly improving the accuracy and robustness of malware detection.
- 4) Experimental validation on diverse malware samples: The proposed method demonstrates superior performance in terms of accuracy and recall compared to

traditional approaches, particularly showcasing strong robustness when handling malware variants.

II. RELATED WORK

A. DEEP LEARNING-BASED MALWARE DETECTION

Recent advancements in deep learning have revolutionized malware detection through automated feature learning. CNNs have demonstrated particular success in processing malware represented as grayscale images [17]. This approach leverages CNNs' ability to capture spatial patterns in binary structures while avoiding manual feature engineering. Subsequent studies have enhanced this paradigm through advanced architectures like ResNet [18] and attention mechanisms [19].

Multimodal learning approaches have emerged as a promising direction, combining static features (e.g., permissions and API calls) with dynamic behavioral features [20]. Hybrid models integrating CNN features with traditional machine learning classifiers have shown improved detection rates for zero-day malware [21]. Recent innovations include graph neural networks for modeling application call graphs [22] and transformer architectures for sequence modeling [23].

Despite these advancements, current methods still face challenges in detecting sophisticated obfuscation techniques. Recent work by [24] demonstrates that frequency domain analysis can reveal patterns obscured by spatial transformations, motivating our integration of Fourier analysis with deep learning.

B. MALWARE FEATURE PATTERNS AND FREQUENCY ANALYSIS

Malware authors increasingly employ sophisticated evasion techniques that create distinct feature patterns. Common strategies include:

- Structural Padding: Insertion of redundant data segments (zero-byte padding, NOP sleds) to alter file signatures [25]
- Code Patterns: Repetitive instruction sequences for anti-debugging or environment detection [26]
- Multi-layer Obfuscation: Combining encryption with code virtualization [27]

These techniques create identifiable frequency signatures. Fourier analysis has proven effective in revealing such patterns through spectral decomposition [28]. Recent studies demonstrate that malware padding generates low-frequency components detectable via Discrete Cosine Transform (DCT) [29], while encrypted payloads exhibit characteristic high-frequency noise [30].

The combination of frequency analysis with deep learning addresses limitations of single-domain approaches. [31] proposed wavelet-CNN fusion for PE file analysis. [32] developed a hybrid model using Short-Time Fourier Transform (STFT) with ResNet-50, showing improved robustness against adversarial examples. Our work extends

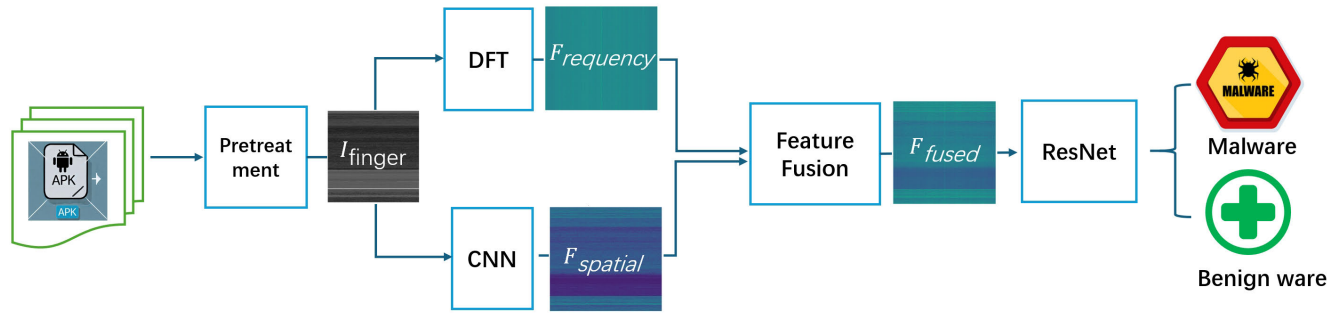


FIGURE 1. Proposed dual-modal feature extraction and fusion framework for Android malware detection. The framework combines frequency domain analysis with Convolutional Neural Networks (CNN), enabling adaptive frequency selection and bidirectional recursive optimization-based feature fusion. This approach captures both global patterns and local details, improving robustness and detection accuracy.

these approaches through adaptive frequency selection and bidirectional feature fusion.

C. HYBRID DETECTION ARCHITECTURES

Recent architectural innovations combine multiple analysis modalities. Reference [33] proposed a three-stream network processing static, dynamic, and frequency features simultaneously. Reference [34] introduced a federated learning framework combining local CNN processing with global frequency analysis. These hybrid approaches demonstrate superior performance compared to single-modality models, particularly for detecting polymorphic malware [35].

ASB shibghatullah et al. have comprehensively analyzed the intrusion detection system based on machine learning [36], which also reflects the trend of similar hybrid architecture exploration in the field of intrusion detection, and provides a reference for us to adopt hybrid architecture in malware detection.

III. METHODOLOGY

A. FRAMEWORK OVERVIEW

The proposed method leverages the spatial and frequency domain features of Android APK files to generate a "fingerprint image" of the malware, enabling a more comprehensive and robust malware detection process through dual-modal feature fusion. The overall framework consists of the following modules: data preprocessing, fingerprint image generation, frequency domain feature extraction, spatial feature extraction, feature fusion, and the classifier. The specific roles and processing steps of each module are shown in Figure 1.

The data preprocessing and fingerprint image generation module converts the binary data of APK files into fingerprint images. The fingerprint image, denoted as $I_{\text{fingerprint}}$, is transformed into a frequency domain image $F(u, v)$ using the Discrete Fourier Transform (DFT) to extract spectral information. The results of the Fourier Transform reflect the periodicity and frequency patterns in the image, aiding in the detection of repetitive patterns and padding data

in malware. From the spectrum, high-frequency and low-frequency components are extracted to capture the global patterns and fine-grained details of the image, respectively. The resulting frequency domain feature representation is denoted as $F_{\text{frequency}}$.

The spatial feature extraction module (CNN) extract spatial features from the fingerprint image, capturing the local characteristics and overall structure of the malware fingerprint image. ResNet is chosen as the backbone network, leveraging multi-layer convolutional operations to progressively extract features, resulting in the spatial domain feature representation F_{spatial} .

The feature fusion module integrates the frequency domain features $F_{\text{frequency}}$ and spatial domain features F_{spatial} , fully utilizing the dual-modal characteristics of malware. At the feature fusion layer, the two features are concatenated and fused using a bidirectional recursive optimization method for spatial and frequency domain feature integration. This approach produces a more discriminative fused feature representation F_{fused} . A channel attention mechanism is introduced to dynamically adjust the weights of frequency and spatial domain features based on the characteristics of each input sample, enhancing the model's focus on critical features.

The fused feature F_{fused} is then passed through a fully connected layer to generate a fixed-length feature vector h . Finally, a Softmax layer performs binary classification to predict the malware detection results (malicious or benign).

B. DATA PREPROCESSING AND FINGERPRINT IMAGE GENERATION

Android malware typically exhibits specific patterns in code, resources, and permission configurations. To fully leverage these characteristics, we adopted an efficient preprocessing and fingerprint image generation method that maps the core components of APK files into grayscale images, enabling feature extraction through CNN [37]. This the method focuses on critical feature regions by selecting DEX files as the primary feature source, which highlights malware code

patterns and structures, improving detection effectiveness. Additionally, to enhance adversarial robustness, a random occlusion method is employed to simulate malware's obfuscation or concealment behavior [38], enabling the model to retain strong detection capabilities even under incomplete information.

1) GRAYSCALE IMAGE GENERATION BASED ON CODE SEGMENTS

APK files contain multiple components, with DEX files (which include the code) being the key region for malware detection. Malware code segments often feature specific instruction patterns, repeated code fragments, or encrypted blocks. Based on this characteristic, we extracted DEX files from APK files and transformed their content into grayscale images. The steps are as follows:

- 1) DEX File Extraction: Extract the DEX file from the APK file as the primary object for feature extraction.
- 2) Byte-to-Grayscale Mapping: Map the byte stream of the DEX file directly into a grayscale image. Assuming the DEX file has a size of N bytes, the byte array $\{b_1, b_2, \dots, b_N\}$ is converted into grayscale values, where each byte b_i corresponds to a pixel with a grayscale value $b_i \in [0, 255]$.
- 3) Image Size Normalization: Standardize the input image dimensions by reshaping the byte stream into a fixed 2D matrix (e.g., $H \times W$). If the byte count N exceeds $H \times W$, the first $H \times W$ bytes are used; if it is smaller, zero-padding is applied.
- 4) Result: After this process, the resulting grayscale image $I_{\text{DEX}} \in \mathbb{R}^{H \times W}$ serves as the "fingerprint image" of the malware, capturing the binary features of the malware code segment.

2) DATA AUGMENTATION: RANDOM OCCLUSION

Random occlusion is employed to simulate various scenarios of hidden or obfuscated malware features. This approach helps train the model to make accurate detections under incomplete information [38]. The occlusion ratio is dynamically adjusted throughout the training process by introducing a control parameter that varies with training progress. Initially, a smaller occlusion ratio is used to allow the model to capture primary features, which is then gradually increased to encourage the model to learn how to make accurate predictions with less information.

The occlusion ratio $p(t)$ is initialized within a range $[p_{\min}, p_{\max}]$, where:

- $p_{\min}$: Minimum occlusion ratio at the beginning of training.
- $p_{\max}$: Maximum occlusion ratio at the end of training.

For each training epoch t , the occlusion ratio $p(t)$ is calculated as a linear function:

$$p(t) = p_{\min} + (p_{\max} - p_{\min}) \cdot \frac{t}{T} \quad (1)$$

where: - $t \in [0, T]$ is the current training epoch. - T is the total number of training epochs.

The size of the occluded region R is dynamically determined based on $p(t)$ as:

$$R = N \times p(t) \quad (2)$$

where N is the total number of pixels in the image. The location, size, and fill color (e.g., white or black) of the occluded region are selected randomly during each training iteration, allowing the model to adapt to varying degrees of occlusion.

C. FOURIER ANALYSIS FOR FREQUENCY DOMAIN FEATURE EXTRACTION

1) MATHEMATICAL REPRESENTATION OF FOURIER TRANSFORM

Given a spatial domain image $f(x, y)$, its two-dimensional Fourier Transform is defined as:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)} \quad (3)$$

where:

- $F(u, v)$ represents the value of the frequency domain image (i.e., the spectrum) at frequency (u, v) .
- $f(x, y)$ represents the grayscale value of the spatial domain image at pixel location (x, y) .
- M and N are the width and height of the image, respectively.
- j is the imaginary unit ($j = \sqrt{-1}$).

The Fourier Transform maps pixel information from the spatial domain to the frequency domain, producing a spectrum $F(u, v)$ that represents the amplitude and phase information of the image's various frequency components.

2) PROCESS OF FREQUENCY DOMAIN FEATURE EXTRACTION

Traditional Fourier Transform uses fixed frequency thresholds. However, malware samples may exhibit different behaviors in varying frequency ranges, requiring dynamic adjustment of frequency thresholds to help the model capture malware features more accurately. The core improvement of adaptive frequency selection lies in dynamically calculating adaptive frequency thresholds. Adaptive frequency selection adjusts the boundary between low and high frequencies dynamically by considering local image features, including local variance and local spectral entropy. The process involves the following four steps:

1. Local Variance Analysis

For malware samples, certain regions may exhibit higher variability, indicating these regions contain more potential malicious behaviors. In these regions, the boundary between low and high frequencies may need to be adjusted more precisely. Regions with higher local variance often require more high-frequency information to capture details, while regions with lower local variance can rely on lower-frequency

components to represent global patterns. Local variance is expressed as:

$$\sigma(I_{\text{local}}) = \frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N (I_{\text{local}}(i, j) - \mu_{\text{local}})^2 \quad (4)$$

where:

- μ_{local} is the mean value of the local region.
- $M \times N$ is the size of the local region.
- $I_{\text{local}}(i, j)$ represents the pixel value in the local region.

2. Local Spectral Entropy

Spectral entropy measures the complexity of frequency domain information and describes the balance of frequency component distribution. Regions with higher spectral entropy indicate more complex or chaotic information, which may be critical to malware behaviors. Regions with lower entropy, by contrast, typically represent smoother, more regular content and are better represented by low-frequency components. The spectral entropy is calculated as:

$$H = - \sum_k p(k) \log p(k) \quad (5)$$

where $p(k)$ represents the probability of the k -th frequency component, indicating the proportion of that frequency's intensity relative to all frequencies.

3. Dynamic Threshold Calculation

Based on the local variance and spectral entropy of the image, the frequency split point r is dynamically adjusted. Specifically, the threshold can be determined by combining local variance and spectral entropy. A higher local variance or spectral entropy suggests the region should include more high-frequency components, as these help reveal detailed features. The dynamic threshold calculation is given as:

$$r = \alpha \cdot \sigma(I_{\text{local}}) + \beta \cdot H \quad (6)$$

where:

- α and β are preset weighting coefficients balancing the influence of local variance and spectral entropy on frequency splitting.

4. Frequency Decomposition and Selection

The image spectrum is divided into low-frequency and high-frequency components based on r . The low-frequency region of the spectrum generally contains the global patterns of the image, suitable for representing its macrostructure, while the high-frequency region includes more fine-grained local features, capturing details and potential malicious patterns. The Low-frequency components and High-frequency components is defined as:

$$F_{\text{low}}(u, v) = F_{\text{centered}}(u, v) \quad \text{if} \quad u^2 + v^2 \leq r^2 \quad (7)$$

$$F_{\text{high}}(u, v) = F_{\text{centered}}(u, v) \quad \text{if} \quad u^2 + v^2 > r^2 \quad (8)$$

By employing adaptive frequency selection, the boundary between low and high frequencies can be dynamically adjusted according to the local characteristics of the image. This method enables the model to adapt flexibly to different types of malware and enhances its robustness against

adversarial samples. Specifically, the calculation of local variance and spectral entropy provides a dynamic basis for frequency splitting, avoiding the limitations of fixed thresholds, which might result in insufficient or overly sensitive feature capture. This improves the accuracy and stability of malware detection.

D. RECURSIVE FEATURE FUSION

In this section, we present the recursive feature fusion framework, which integrates spatial and frequency domain features through a multi-stage process. The framework includes three core components: Attention-Guided Pre-Fusion, Frequency-Guided Adjustment of Spatial Features, and Recursive Optimization. These components work synergistically to enhance feature representation by leveraging complementary information from both domains. Below, we describe each component in detail.

1) ATTENTION-GUIDED PRE-FUSION

Before fusing spatial and frequency domain features, we introduce an attention mechanism to prioritize informative regions and suppress noise. This step ensures that the fusion process focuses on the most discriminative features. The attention-guided pre-fusion is formulated as follows:

1. Attention Map Generation: For spatial domain features $\mathbf{X}_s \in \mathbb{R}^{n \times d}$, we compute an attention map $\mathbf{A}_s$ using a learnable convolutional layer followed by a sigmoid activation:

$$\mathbf{A}_s = \sigma(\mathbf{W}_a * \mathbf{X}_s + \mathbf{b}_a) \quad (9)$$

where $\mathbf{W}_a$ and $\mathbf{b}_a$ are learnable parameters, $*$ denotes convolution, and $\sigma(\cdot)$ is the sigmoid function.

2. Feature Weighting: The spatial features are weighted by the attention map to emphasize critical regions:

$$\mathbf{X}_s^{\text{att}} = \mathbf{A}_s \odot \mathbf{X}_s \quad (10)$$

where $\odot$ denotes element-wise multiplication.

3. Pre-Fusion: The attention-weighted spatial features $\mathbf{X}_s^{\text{att}}$ are combined with frequency domain features $\mathbf{X}_f$ using a preliminary fusion strategy:

$$\mathbf{X}_{\text{pre-fused}} = \beta \mathbf{X}_s^{\text{att}} + (1 - \beta) \mathbf{X}_f \quad (11)$$

where $\beta \in [0, 1]$ is a balancing parameter.

2) FREQUENCY-GUIDED ADJUSTMENT OF SPATIAL FEATURES

To further refine spatial features using frequency domain insights, we propose a frequency-guided adjustment module. This step aligns spatial features with global frequency patterns to enhance their robustness.

1. Frequency-Based Modulation: Given the frequency domain features $\mathbf{X}_f$, we compute a modulation vector $\mathbf{M} \in \mathbb{R}^d$ by averaging frequency components across spatial

dimensions:

$$\mathbf{M} = \frac{1}{n} \sum_{i=1}^n \mathbf{X}_f^{(i)} \quad (12)$$

where $\mathbf{X}_f^{(i)}$ denotes the i -th sample in $\mathbf{X}_f$.

2. Feature Adjustment: The spatial features $\mathbf{X}_s^{att}$ are adjusted using the modulation vector:

$$\mathbf{X}_s^{adj} = \mathbf{X}_s^{att} + \gamma \cdot \mathbf{M} \cdot \mathbf{X}_s^{att} \quad (13)$$

where γ is a scaling factor that controls the influence of frequency guidance.

3. Adjusted Fusion: The adjusted spatial features $\mathbf{X}_s^{adj}$ are fused with frequency features $\mathbf{X}_f$:

$$\mathbf{X}_{fused} = \alpha \mathbf{X}_s^{adj} + (1 - \alpha) \mathbf{X}_f \quad (14)$$

where $\alpha \in [0, 1]$ is the fusion weight.

3) RECURSIVE OPTIMIZATION PROCESS

The fused features $\mathbf{X}_{fused}$ are further refined through a recursive optimization process to minimize a loss function that balances reconstruction accuracy and sparsity. The optimization steps are as follows:

1. Loss Function:

$$\mathcal{L}(\mathbf{X}_{fused}) = \|\mathbf{Y} - \mathbf{X}_{fused}\|_2^2 + \lambda \|\mathbf{X}_{fused}\|_1 \quad (15)$$

where $\mathbf{Y}$ is the target output, λ controls sparsity, and $\|\cdot\|_2^2$, $\|\cdot\|_1$ denote the squared Euclidean norm and L1 norm, respectively.

2. Gradient Descent Update: The features are updated iteratively using:

$$\mathbf{X}_{fused}^{(t+1)} = \mathbf{X}_{fused}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{X}_{fused}^{(t)}) \quad (16)$$

where η is the learning rate, and the gradient $\nabla \mathcal{L}$ is computed as:

$$\nabla \mathcal{L}(\mathbf{X}_{fused}^{(t)}) = -2(\mathbf{Y} - \mathbf{X}_{fused}^{(t)}) + \lambda \cdot \text{sign}(\mathbf{X}_{fused}^{(t)}) \quad (17)$$

3. Convergence Check: The process terminates when the loss change between iterations falls below a threshold ϵ :

$$\|\mathcal{L}(\mathbf{X}_{fused}^{(t+1)}) - \mathcal{L}(\mathbf{X}_{fused}^{(t)})\| < \epsilon \quad (18)$$

4) ASSUMPTIONS AND ALGORITHM SUMMARY

In this work, we propose a novel feature fusion method guided by attention and frequency information, aiming to optimize the integration of spatial and frequency features. Our approach is based on the following assumptions: spatial and frequency features provide complementary information, the attention mechanism can accurately identify discriminative regions, and L1 regularization enhances interpretability and generalizability.

The attention-guided pre-fusion step suppresses noisy or irrelevant spatial features before fusion, reducing their potential interference. The frequency-guided adjustment aligns spatial features with global frequency patterns, thereby improving robustness to local distortions. Finally, the

recursive optimization process fine-tunes the fused features by balancing reconstruction fidelity and sparsity, ensuring adaptability to downstream tasks.

As described in Algorithm 1, the proposed method integrates spatial and frequency features through a recursive fusion process guided by attention and frequency information.

Algorithm 1 Recursive Feature Fusion With Attention and Frequency Guidance

- 1: **Input:** Spatial features $\mathbf{X}_s$, frequency features $\mathbf{X}_f$, target $\mathbf{Y}$, parameters $\alpha, \beta, \gamma, \lambda, \eta, \epsilon$
- 2: **Output:** Optimized fused features $\mathbf{X}_{fused}$
- 3: **Attention Pre-Fusion:**
- 4: Compute attention map $\mathbf{A}_s$.
- 5: Generate $\mathbf{X}_s^{att} = \mathbf{A}_s \odot \mathbf{X}_s$.
- 6: Pre-fuse: $\mathbf{X}_{pre-fused} = \beta \mathbf{X}_s^{att} + (1 - \beta) \mathbf{X}_f$.
- 7: **Frequency-Guided Adjustment:**
- 8: Compute modulation vector $\mathbf{M}$.
- 9: Adjust spatial features: $\mathbf{X}_s^{adj} = \mathbf{X}_s^{att} + \gamma \cdot \mathbf{M} \cdot \mathbf{X}_s^{att}$.
- 10: Fuse adjusted features: $\mathbf{X}_{fused}^{(0)} = \alpha \mathbf{X}_s^{adj} + (1 - \alpha) \mathbf{X}_f$.
- 11: **Recursive Optimization:**
- 12: Initialize $t = 0$.
- 13: **while** not converged **do**
- 14: Compute loss $\mathcal{L}(\mathbf{X}_{fused}^{(t)})$.
- 15: Update features: $\mathbf{X}_{fused}^{(t+1)} = \mathbf{X}_{fused}^{(t)} - \eta \nabla \mathcal{L}$.
- 16: Check convergence using ϵ .
- 17: **end while**
- 18: **Return** $\mathbf{X}_{fused}^{(t)}$

E. CLASSIFICATION WITH FULLY CONNECTED NEURAL NETWORKS

The fused feature $\mathbf{F}_{fused} \in \mathbb{R}^{H \times W \times C}$, which integrates spatial and frequency domain information, encapsulates rich global patterns and local details. This high-dimensional feature can be directly classified using a fully connected neural network (DNN), consisting of three main components: input layer, hidden layers, and output layer.

Input Layer: The fused feature is flattened into a one-dimensional vector:

$$\mathbf{F}_{fused}^{flat} \in \mathbb{R}^d, \quad d = H \times W \times C \quad (19)$$

Hidden Layers: The flattened feature is passed to two fully connected hidden layers, each progressively reducing the dimensionality. The first hidden layer reduces the feature dimension from d to $d/2$. The second hidden layer further reduces it to $d/4$. Each layer uses ReLU activation to introduce non-linearity and includes Dropout (with a probability of 0.5) to mitigate overfitting.

Output Layer: The output layer employs the Softmax activation function to perform binary classification, predicting the probability of the sample being malicious or benign:

$$\hat{y} = \text{Softmax}(\mathbf{W}_{out} \cdot \mathbf{h} + \mathbf{b}) \quad (20)$$

where $\mathbf{h}$ is the output from the second hidden layer, and $\mathbf{W}_{out}$ and $\mathbf{b}$ are the weights and biases of the output layer.

Loss Function: To optimize the classifier's performance, the cross-entropy loss function is used:

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i) \quad (21)$$

where y_i and $\hat{y}_i$ represent the true label and the predicted probability for class i , respectively.

Optimization:

- The Adam optimizer is employed for efficient convergence, combined with a learning rate decay strategy to accelerate convergence and improve performance.

- L2 regularization is applied to constrain weights, further reducing overfitting.

- Dropout is used throughout to enhance generalization capability.

By combining these techniques, the proposed classification framework ensures accurate and robust detection of malicious samples.

IV. EXPERIMENTS AND RESULTS

A. DATASETS AND EXPERIMENTAL SETUP

To verify the applicability and robustness of the proposed method in different scenarios, multiple widely used public datasets were selected for experiments. Drebin Dataset [3]: Selected 5560 malicious samples and 6000 benign samples. AndroZoo Dataset [39]: Selected 6000 malicious samples and 6000 benign samples. To ensure fairness and computational feasibility, an equal number of malicious and benign samples were randomly selected from the AndroZoo dataset. The total number of malicious samples is approximately 10,000. Malgenome Dataset [40]: Contains 1200 malware samples, covering various malware families. (Since this dataset includes only malware samples, we supplemented it with approximately 1300 benign samples from the AndroZoo dataset to form a complete dataset). AMD Dataset [41]: Contains 2460 malicious samples and 1620 benign samples.

Additionally, to enhance the scientific rigor and generalizability of the experiments, 500 benign samples and 500 malicious samples were separately selected from the AndroZoo dataset as an additional test set. All APK files were processed into fingerprint images and normalized to 224×224 grayscale images. ResNet-50 was used as the backbone network for spatial feature extraction. The initial learning rate was set to 0.001, and the weight decay parameter was set to 0.0005. Hardware: NVIDIA A100 GPU. Software environment: Python 3.9, PyTorch 2.0, CUDA 11.6. Operating system: Ubuntu 20.04.

B. PERFORMANCE METRICS

Model performance is typically evaluated using a variety of metrics, including Accuracy, Precision, Recall, F1-Score, and AUC-ROC. Accuracy measures the proportion of correctly classified samples, reflecting the overall classification ability of the model. Precision focuses on the proportion of predicted

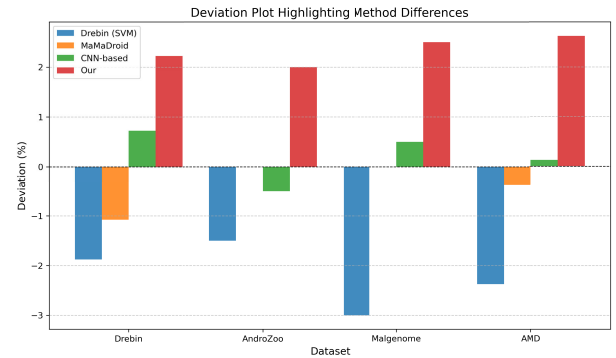


FIGURE 2. Deviation plot highlighting method differences.

malware samples that are actually malware, assessing the model's ability to reduce false positives (FP). Recall evaluates the proportion of actual malware samples correctly identified, reflecting the model's detection capability, especially in scenarios where malware samples are sparse. F1-Score, as the harmonic mean of Precision and Recall, balances the trade-off between the model's precision and detection capability, making it particularly suitable for tasks with imbalanced classes. AUC-ROC assesses the relationship between the False Positive Rate (FPR) and the True Positive Rate (TPR), providing an overall evaluation of the model's ability to distinguish between positive and negative samples. The closer the AUC-ROC value is to 1, the better the model performance. These metrics comprehensively reflect the model's detection capability and robustness from different dimensions, providing reliable evaluation criteria for optimizing malware detection models.

C. PERFORMANCE COMPARISON

To ensure comprehensive benchmarking, we compared our method with three representative approaches: *Drebin (SVM)* [3], a classical static analysis method using permissions and API calls with SVM classifiers; *MaMaDroid* [22], which models API call sequences via Markov chains for behavior pattern analysis; *CNN-based approaches* [42], which convert malware binaries into grayscale images for convolutional feature extraction. These baselines span traditional machine learning, sequence modeling, and deep learning paradigms, covering the mainstream technical routes in Android malware detection.

As shown in Figure 2, the proposed method achieves superior performance across all four datasets in terms of Accuracy, Precision, and AUC-ROC. In all datasets, the yellow bars (proposed method) are significantly higher than those of other methods, indicating that the proposed approach outperforms the mean performance and demonstrates a clear advantage across these datasets.

On the Malgenome dataset, some methods may perform close to or slightly above the mean in certain metrics, but overall, they do not exceed the deviation range of the proposed method. While other methods might achieve

TABLE 1. Performance on different datasets.

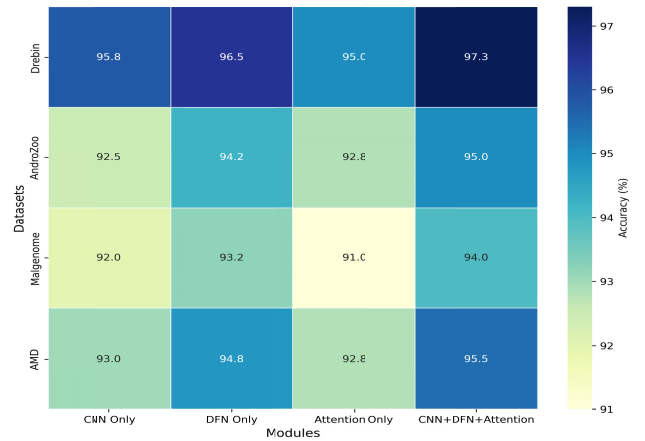
Dataset	Method	Accuracy	Precision	Recall	F1 Score	AUC-ROC
Drebin	Drebin (SVM)	93.2	91.5	92.0	91.8	94.0
	MaMaDroid	94.0	92.3	93.5	92.9	95.5
	CNN-based	95.8	94.0	94.5	94.2	96.8
	Our	97.3	96.0	96.5	96.2	98.5
AndroZoo	Drebin (SVM)	91.5	89.0	89.5	89.3	92.5
	MaMaDroid	93.0	91.2	91.5	91.4	94.5
	CNN-based	92.5	90.2	91.0	90.6	93.0
	Our	95.0	93.5	93.8	93.6	96.2
Malgenome	Drebin (SVM)	88.5	85.5	86.0	85.8	90.0
	MaMaDroid	91.5	90.2	90.0	90.1	92.0
	CNN-based	92.0	91.0	91.5	91.2	93.5
	Our	94.0	92.5	93.0	92.7	95.2
AMD	Drebin (SVM)	90.5	88.0	88.5	88.2	91.0
	MaMaDroid	92.5	91.0	91.5	91.2	93.5
	CNN-based	93.0	91.0	91.5	91.2	94.0
	Our	95.5	94.0	94.2	94.1	96.8

comparable or slightly better results on specific datasets, they are generally outperformed by the proposed approach.

As shown in Table 1, the proposed method demonstrates excellent performance across multiple datasets.

The proposed method demonstrates superior performance across multiple datasets, showcasing its robustness and effectiveness. On the Drebin dataset, it achieves an accuracy of 97.3% and an AUC-ROC of 98.5%, significantly outperforming MaMaDroid (94.0%, 95.5%) and CNN-based methods (95.8%, 96.8%). In the real-world AndroZoo dataset, the proposed method achieves 95.0% accuracy, surpassing MaMaDroid (93.0%) and CNN-based methods (92.5%) by effectively utilizing dual-modal feature fusion. For the small Malgenome dataset, it achieves 94.0% accuracy, higher than MaMaDroid (91.5%) and CNN-based methods, despite a slightly lower AUC-ROC (92.8%) compared to CNN-based (93.5%). On the AMD dataset, the proposed method achieves the highest performance with an accuracy of 95.5% and an AUC-ROC of 96.8%, outperforming CNN-based (93.0%) and DFN (92.8%). These results highlight the proposed method's ability to leverage dual-modal feature fusion and attention mechanisms, demonstrating consistent advantages across diverse datasets.

Technical Differentiation Analysis: The superiority of our method stems from its dual-modal fusion mechanism, which addresses the limitations of the baselines: MaMaDroid's reliance on API call sequences [22] limits its ability to capture spatial features from code structures, whereas our visual fingerprinting captures both global and local patterns. Traditional CNN-based methods [42] process single-modal image inputs, lacking cross-modal interaction between semantic and structural features. Our attention-guided fusion module enables adaptive weighting of features from APK semantics and control flow graphs. Compared to Drebin's handcrafted features [3], our automated feature learning reduces human bias and adapts to evolving malware variants.

**FIGURE 3.** ACC Heatmap.

D. ABLATION STUDY

To evaluate the contribution of each module in the proposed method to detection performance, we conducted an ablation study to analyze their impact on overall performance. This validates the contributions of dual-modal feature extraction, feature fusion, and the attention mechanism. The heatmap Figure 3 visually illustrates the accuracy achieved with individual modules and the complete method, where darker colors indicate higher accuracy. The combination of CNN+DFN+Attention has the darkest colors across all datasets, showcasing its optimal overall performance and reflecting the synergistic effects of the fusion module.

1) GLOBAL COMPARATIVE ANALYSIS

The ablation study reveals that individual modules have varying effectiveness in malware detection. Domain Fusion Network (DFN) Only outperforms CNN Only and Attention Only, particularly on complex datasets like Malgenome

TABLE 2. Ablation experiment results.

Module	Dataset	Accuracy	Precision	Recall	F1 Score	AUC-ROC
CNN Only	Drebin	95.8	94.0	94.5	94.2	96.8
	AndroZoo	92.5	90.2	91.0	90.6	93.0
	Malgenome	92.0	91.0	91.5	91.2	93.5
	AMD	93.0	91.0	91.5	91.2	94.0
DFN Only	Drebin	96.5	95.0	95.2	95.1	97.5
	AndroZoo	94.2	92.5	92.8	92.6	95.8
	Malgenome	93.2	92.0	92.2	92.1	94.7
	AMD	94.8	93.0	93.2	93.1	96.0
Attention Only	Drebin	95.0	93.8	93.5	93.6	96.0
	AndroZoo	92.8	91.0	90.8	90.9	94.2
	Malgenome	91.0	90.5	90.2	90.3	92.8
	AMD	92.8	91.8	91.0	91.4	93.8
CNN+DFN+Attention	Drebin	97.3	96.0	96.5	96.2	98.5
	AndroZoo	95.0	93.5	93.8	93.6	96.2
	Malgenome	94.0	92.5	93.0	92.7	95.2
	AMD	95.5	94.0	94.2	94.1	96.8

TABLE 3. Adversarial experiment results.

Dataset	Attack Method	Accuracy	Precision	Recall	F1 Score	AUC-ROC
Drebin	Drebin (SVM)	72.5	70.2	71.0	70.6	73.0
	MaMaDroid	78.0	75.5	76.0	75.8	78.5
	CNN-based	83.5	81.0	81.5	81.2	84.0
	Our	89.5	87.0	87.5	87.2	90.0
AndroZoo	Drebin (SVM)	70.2	68.5	69.0	68.7	71.0
	MaMaDroid	75.0	72.0	73.0	72.5	76.0
	CNN-based	81.5	79.0	79.5	79.2	83.0
	Our	87.0	85.5	85.8	85.6	88.5
Malgenome	Drebin (SVM)	68.0	65.5	66.0	65.7	69.0
	MaMaDroid	73.5	70.5	71.0	70.7	75.0
	CNN-based	80.0	77.0	77.5	77.2	81.0
	Our	85.5	83.0	83.5	83.2	86.0
AMD	Drebin (SVM)	71.5	69.0	69.5	69.2	72.0
	MaMaDroid	76.0	74.0	73.5	73.7	77.0
	CNN-based	83.0	81.0	80.5	80.7	84.0
	Our	88.5	86.5	87.0	86.7	90.0

and AMD, with notable improvements in Precision and Recall, highlighting its strength in handling complex features. Attention Only performs close to CNN Only on simpler datasets like Drebin and AndroZoo but is weaker overall due to its inability to fully extract low-level features when used alone.

The combination of CNN+DFN+Attention achieves the best performance across all datasets, with Accuracy reaching 97.3% and AUC-ROC 98.5% on the Drebin dataset. This demonstrates the synergistic effect of combining modules, optimizing feature extraction and weighting, and validating the effectiveness of the proposed design.

The proposed method demonstrates exceptional performance across multiple datasets. On the Drebin dataset, it achieved 97.3% accuracy and 98.5% AUC-ROC, significantly outperforming MaMaDroid (94.0%, 95.5%) and CNN-based methods (95.8%, 96.8%).

2) FINE-GRAINED ANALYSIS

The results highlight the contributions of each module and their synergy:

Individual Contributions: CNN excels in Accuracy and AUC-ROC, achieving 96.8% AUC-ROC on the Drebin dataset. DFN enhances Precision and Recall, reaching 92.5% and 92.8% on the AndroZoo dataset, outperforming other single modules.

Attention improves Recall when combined with CNN and DFN, achieving 96.5% on the Drebin dataset.

Synergistic Effects: Combining CNN, DFN, and Attention optimizes feature extraction and weighting, achieving a Recall of 96.5% and AUC-ROC of 98.5% on Drebin.

Complex Dataset Performance: On Malgenome and AMD, the combined modules adapt to complex features and noise, improving Recall and F1 Score to 93% and 92.7% on Malgenome, and 94.2% and 94.1% on AMD.

The study validates that combining CNN, DFN, and Attention significantly improves performance, particularly on complex datasets and in adversarial scenarios, showcasing strong robustness and adaptability.

E. ADVERSARIAL TESTING

To validate the robustness of the proposed method (Our) under FGSM (Fast Gradient Sign Method) attacks, both global and fine-grained analyses confirm its significant resilience. As shown in Table 3, the proposed method demonstrates superior robustness under FGSM attacks. This advantage stems from the combined improvements in model design, training strategies, and feature distribution optimization.

The proposed method (Our) demonstrates strong robustness against FGSM attacks across datasets. On the Drebin dataset, it achieves 89.5% Accuracy (+6% over CNN-based) and an AUC-ROC of 90, balancing Precision and Recall for effective malicious sample detection. On AndroZoo, it improves AUC-ROC from 83 to 88.5 and F1 Score from 79.2 to 85.6, showcasing resilience to adversarial perturbations. On Malgenome, it achieves the highest Recall (83.5) and AUC-ROC (86) with Precision at 83.0, maintaining robustness under high attack intensity. On AMD, it achieves 88.5% Accuracy and 90 AUC-ROC, with Precision and Recall at 86.5 and 87, demonstrating adaptability in complex environments. These results highlight the effectiveness of adversarial training, feature space optimization, and structural improvements in enhancing resilience to adversarial attacks.

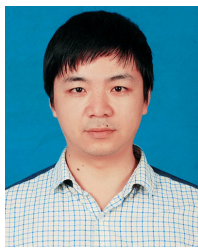
V. CONCLUSION

This paper proposes a dual-modal malware detection method that combines spatial and frequency domain features, leveraging the Fourier transform to extract frequency domain features to capture both global patterns and local details of malware. The use of adaptive frequency partitioning dynamically adjusts high- and low-frequency information, further enhancing feature extraction accuracy. The feature fusion module, integrated with an attention mechanism, effectively combines frequency and spatial domain features, improving the model's discriminative capability and robustness. Experimental results demonstrate that the proposed method significantly outperforms existing approaches across multiple datasets, particularly in detecting complex malware patterns and handling adversarial samples. The effectiveness of frequency domain feature extraction and fusion provides an innovative solution for malware detection.

REFERENCES

- [1] Statista, GmbH. (2023). *Mobile Operating Systems' Market Share Worldwide From 2012 To 2023*. Accessed: Dec. 1, 2023. [Online]. Available: <https://www.statista.com/statistics/>
- [2] AV-TEST GmbH. (Jul. 2023). *Android Malware Landscape 2023*. [Online]. Available: <https://www.av-test.org/>
- [3] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck, "Drebin: Effective and explainable detection of Android malware in your pocket," in *Proc. NDSS*, vol. 14, no. 1, 2014, pp. 23–26.
- [4] K. Allix, T. F. Bissyandé, J. Klein, and Y. L. Traon, "AndroZoo: Collecting millions of Android apps for the research community," in *Proc. IEEE/ACM 13th Work. Conf. Mining Softw. Repositories (MSR)*, May 2016, pp. 468–471.
- [5] S. H. Hamed and A. S. Albahri, "Unlocking the potential of autism detection: Integrating traditional feature selection and machine learning techniques," *Appl. Data Sci. Anal.*, vol. 2023, pp. 42–58, May 2023.
- [6] K. Tam, A. Feizollah, N. B. Anuar, R. Salleh, and L. Cavallaro, "The evolution of Android malware and Android analysis techniques," *ACM Comput. Surv.*, vol. 49, no. 4, pp. 1–41, Dec. 2017.
- [7] M. K. Alzaylaee, S. Y. Yerima, and S. Sezer, "DL-droid: Deep learning based Android malware detection using real devices," *Comput. Secur.*, vol. 89, Feb. 2020, Art. no. 101663.
- [8] K. Grosse, N. Papernot, P. Manoharan, M. Backes, and P. McDaniel, "Adversarial examples for malware detection," in *Proc. 22nd Eur. Symp. Res. Comput. Secur. Comput. Security (ESORICS)*, Oslo, Norway. Cham, Switzerland: Springer, Sep. 2017, pp. 62–79.
- [9] L. Li, Q. Li, T. F. Bissyandé, and J. Klein, "AdvDroid: A framework for Android malware detection based on data augmentation and deep learning," in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun.*, Oct. 2019, pp. 50–57.
- [10] D.-J. Wu, C.-H. Mao, T.-E. Wei, H.-M. Lee, and K.-P. Wu, "DroidMat: Android malware detection through manifest and API calls tracing," in *Proc. 7th Asia Joint Conf. Inf. Secur.*, Aug. 2012, pp. 62–69.
- [11] E. B. Karbab, M. Debbabi, A. Derhab, and D. Mouheb, "MalDozer: Automatic framework for Android malware detection using deep learning," *Digit. Invest.*, vol. 24, pp. S48–S59, Mar. 2018.
- [12] Y. Aafer, W. Du, and H. Yin, "Droidapiminer: Mining api-level features for robust malware detection in android," in *Proc. 9th Int. Conf. Secur. Privac Commun. Syst.*, Sydney, NSW, Australia, 2013, pp. 86–103.
- [13] M. Lindorfer, M. Neugschwandtner, and C. Platzer, "MARVIN: Efficient and comprehensive mobile app classification through static and dynamic analysis," in *Proc. IEEE 39th Annu. Comput. Softw. Appl. Conf.*, vol. 2, Jul. 2015, pp. 422–433.
- [14] B. Davis, T. Chen, A. Dhavle, Y. Shoshitaishvili, and A. Doupé, "Android dynamic analysis: A systematic literature review," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 184:1–184:35, 2021.
- [15] L. Chen, J. Zhang, J. Ren, and Y. Yang, "Malicious Android application detection using permission and behavior features," *J. Inf. Secur. Appl.*, vol. 35, pp. 91–104, Jun. 2017.
- [16] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *Proc. 29th Australas. Joint Conf. Adv. Artif. Intell.*, Hobart, TAS, Australia, 2016, pp. 137–149.
- [17] D. Vasan, M. Alazab, S. Wassan, H. Naeem, B. Safaei, and Q. Zheng, "IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture," *Comput. Netw.*, vol. 171, Apr. 2020, Art. no. 107138.
- [18] W. Huang and J. W. Stokes, "MtNet: A multi-task neural network for dynamic malware classification," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*. Cham, Switzerland: Springer, Jun. 2016, pp. 399–418.
- [19] Y. Li, J. Jang, X. Wang, and C. Choi, "Attention-based malware analysis for Android systems," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1899–1912, Mar. 2021.
- [20] N. McLaughlin, J. M. del Rincon, B. Kang, S. Y. Yerima, P. Miller, S. Sezer, Y. Safaei, E. Trickle, Z. Zhao, A. Doupé, and G.-J. Ahn, "Deep Android malware detection," in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy*, 2017, pp. 301–308.
- [21] W. Huang, M. Sun, and Q. Qian, "Hybrid malware detection with multi-domain features," *Comput. Secur.*, vol. 121, Oct. 2022, Art. no. 102839.
- [22] E. Mariconti, L. Onwuzurike, P. Andriotis, E. D. Cristofaro, G. Stringhini, and M. Egele, "MALGRAPH: Android malware detection through system call graphs," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2017, pp. 604–615.
- [23] E. Raff and C. Nicholas, "Malware detection via long-term memory network," *Comput. Secur.*, vol. 103, Nov. 2021, Art. no. 102158.
- [24] Z. Chen, L. Wang, and W. Zhang, "Frequency-aware malware detection with spectral CNN," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2345–2358, 2023.
- [25] B. Anderson, "Structure-informed malware detection through section padding analysis," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, 2018, pp. 302–315.
- [26] I. Yousefi and R. Curtmola, "Periodic execution patterns in malware for environmental perception," in *Proc. ARES 15th Int. Conf. Availability, Rel. Secur.*, 2020, pp. 1–10.

- [27] X. Li, W. Diao, and Y. Zhang, "Multiguard: Hybrid code protection using dynamic encryption and virtualization," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2021–2035, Mar. 2023.
- [28] J. Smith and J. Lee, "Frequency domain analysis of API call sequences for malware classification," in *Proc. IEEE Secur. Privacy*, Oct. 2023, pp. 1–15.
- [29] K. Han, T. Kim, and B. B. Kang, "Detecting malware padding patterns via DCT analysis," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, 2022, pp. 789–801.
- [30] L. Wang, Y. Guo, and X. Chen, "Spectral analysis of encrypted malware payloads," *Comput. Secur.*, vol. 131, Apr. 2023, Art. no. 103288.
- [31] H. Zhang, X. Xiao, and J. Xu, "Wavelet-CNN fusion for malware detection," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, Dec. 2021, pp. 220–227.
- [32] D. Gibert, J. Planes, and C. Mateu, "Hybrid ResNet-STFT architecture for malware detection," *Expert Syst. Appl.*, vol. 198, May 2022, Art. no. 116848.
- [33] A. Singh, P. Li, and Y. Chen, "Multi-stream malware detection with tri-modal features," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 2831–2844, Apr. 2023.
- [34] Y. Liu, Y. Chen, J. Kang, and Q. Li, "Hybrid federated learning: Algorithms and implementation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 12, pp. 5485–5499, Dec. 2020.
- [35] W. Zhang, X. Li, H. Wang, and Q. Lu, "Multimodal federated learning for polymorphic malware detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2163–2177.
- [36] A. S. B. Shibghatullah, "Mitigating developed persistent threats (APTs) through machine learning-based intrusion detection systems: A comprehensive analysis," *SHIFRA*, vol. 2023, pp. 17–25, Mar. 2023.
- [37] X. Xiao and S. Yang, "An image-inspired and CNN-based Android malware detection approach," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2019, pp. 1259–1261.
- [38] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 7, pp. 13001–13008.
- [39] F. A. Aboaoja, A. Zainal, F. A. Ghaleb, B. A. S. Al-Rimy, T. A. E. Eisa, and A. A. H. Elnour, "Malware detection issues, challenges, and future directions: A survey," *Appl. Sci.*, vol. 12, no. 17, p. 8482, Aug. 2022.
- [40] Y. Zhou and X. Jiang, "Dissecting Android malware: Characterization and evolution," in *Proc. IEEE Symp. Secur. Privacy*, May 2012, pp. 95–109.
- [41] F. Wei, S. Roy, S. Y. Yerima, S. Pastrana, J. Ren, R. Crane, and N. Miller, "Deep ground truth analysis of current Android malware," in *Proc. Int. Conf. Passive Act. Netw. Meas. (PAM)*, 2017, pp. 364–374.
- [42] S. Ni, Q. Qian, and R. Zhang, "Feature extraction for mobile malware detection using CNN," in *Proc. IEEE BigData*, Sep. 2020, pp. 4143–4151.



JIAN WANG received the Master of Engineering degree in computer technology from Wuhan University of Technology, Wuhan, China, in 2012. He is currently pursuing the Ph.D. degree in computer application technology with the Faculty of Applied Sciences, Macao Polytechnic University, Macau, SAR, China.

He is also an Associate Professor with Guangzhou Institute of Software, Guangzhou, China. He has successfully led several provincial-level research projects and has actively contributed to the development of innovative solutions in his fields of expertise. He also serves as a peer reviewer for multiple international journals and conferences. His research interests include object detection, federated learning, and edge computing.



XUE HUA LIU received the Master of Engineering degree in computer technology from Wuhan University of Technology, Wuhan, China, in 2012.

She is currently an Associate Professor with the Internet of Things Engineering Program, School of Software Engineering, Guangzhou Institute of Software, Guangzhou, China. She has successfully led multiple provincial-level teaching and research projects. She has also mentored students who have achieved national-level awards in various competitions. Her work significantly contributes to the development of innovative solutions in her field of expertise. Her research interests include object detection and edge computing.



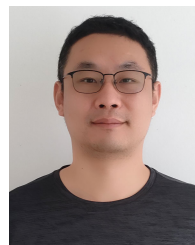
MENG YUAN HUANG is currently pursuing the degree in network engineering with the School of Cyberspace Security, Guangzhou Institute of Software.

During his time at school, he was the Head of the Technology Department of the Network Management Center for one year, where he was recognized for his outstanding leadership and contributions, earning the title of *Excellent Minister*. Academically, he has been highly active in various competitions, achieving notable successes. He secured a place in the top 100 in South China Division of the National College Students' Information Security Competition Practice Competition. Additionally, he has consistently excelled in his studies, receiving multiple student scholarships from the school.



PEI YI ZHOU is currently pursuing the degree in network engineering majoring with the School of Network Technology, Guangzhou Institute of Software, with a specialization in network security technology.

She has actively participated in provincial-level research projects, demonstrating her commitment to advancing knowledge in her field. Additionally, she is dedicated to staying abreast of the latest technological developments, ensuring that her skills and understanding remain at the forefront of network security. Her research interests include malicious code and network security.



YUAN XU received the B.E. degree in automation and the M.S. degree in measurement technology and automation devices from Xi'an Technology University, Shaanxi, China, in 2007 and 2011, respectively. He is currently pursuing the Ph.D. degree in computer applied technology with Macao Polytechnic University, Macau, China. He is also a Lecturer with the College of Artificial Intelligence, Dongguan City University, and Macao Polytechnic University. His main research interests include medical image processing and intelligent devices.

...